

EMSI — École Marocaine des Sciences de l'Ingénieur

Module : Deep Learning

Année universitaire 2025–2026

RAPPORT DE PROJET

Conception, implémentation et analyse de modèles de Deep Learning

MLP • CNN • Seq2Seq • GRU • Architectures hybrides • Agent

Réalisé par	: Rania Abdelali
Encadré par	: Mme. HIDILA Zineb
Filière	: Intelligence artificielle et Science des données
Date de soutenance	: 25 juin 2026

20 juin 2026

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Je remercie tout particulièrement **Mme. HIDILA Zineb**, mon encadrante de projet, pour sa disponibilité, ses précieux conseils et son accompagnement tout au long de ce travail. Son expertise en deep learning et sa rigueur scientifique m'ont été d'une aide précieuse.

Je remercie également l'ensemble du corps enseignant de l'EMSI pour la qualité de leur enseignement et pour m'avoir transmis les connaissances nécessaires à la réalisation de ce projet.

Enfin, je remercie ma famille et mes amis pour leur soutien indéfectible tout au long de cette année universitaire.

Rania Abdelali

Résumé

Ce rapport présente le développement et l'analyse comparative de trois architectures fondamentales de deep learning : le Perceptron Multicouche (MLP) pour la classification de données tabulaires, le réseau de neurones convolutionnel (CNN) pour la classification d'images, et l'architecture Seq2Seq pour la traduction automatique. Chaque modèle a été implémenté avec **PyTorch**, entraîné sur un dataset de référence (Breast Cancer Wisconsin, CIFAR-10, Tatoeba), puis intégré dans une application web interactive développée avec **Flask**. Les résultats obtenus montrent une précision de 96,5% pour le MLP, des performances correctes pour le CNN (36%) compte tenu des contraintes matérielles, et une réduction significative de la loss pour le Seq2Seq (de 8,23 à 1,48). Une analyse critique des résultats, des difficultés rencontrées et des perspectives d'amélioration est également proposée.

Mots-clés : Deep Learning, MLP, CNN, Seq2Seq, GRU, PyTorch, Classification, Traduction automatique, Application web.

Abstract

This report presents the development and comparative analysis of three fundamental deep learning architectures : Multilayer Perceptron (MLP) for tabular data classification, Convolutional Neural Network (CNN) for image classification, and Seq2Seq architecture for automatic translation. Each model was implemented with PyTorch, trained on reference datasets (Breast Cancer Wisconsin, CIFAR-10, Tatoeba), and then integrated into an interactive web application developed with Flask. The results show 96.5% accuracy for the MLP, decent performance for the CNN (36%) given hardware constraints, and a significant loss reduction for Seq2Seq (from 8.23 to 1.48). A critical analysis of the results, encountered difficulties, and improvement perspectives is also provided.

Keywords : Deep Learning, MLP, CNN, Seq2Seq, GRU, PyTorch, Classification, Machine Translation, Web application.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
1 Introduction	10
1.1 Contexte général	10
1.2 Problématique	10
1.3 Objectifs du projet	10
1.4 Organisation du rapport	11
2 Préparation des données	11
2.1 Introduction	11
2.2 Dataset 1 : Breast Cancer Wisconsin	11
2.2.1 Présentation du dataset	11
2.2.2 Analyse exploratoire	12
2.2.3 Prétraitement	12
2.3 Dataset 2 : CIFAR-10	12
2.3.1 Présentation du dataset	12
2.3.2 Prétraitement	13
2.4 Dataset 3 : Tatoeba (Français-Anglais)	13
2.4.1 Présentation du dataset	13
2.4.2 Prétraitement	14
3 Partie I : MLP - Classification du cancer du sein	14
3.1 Fondements théoriques	14
3.1.1 Le Perceptron Multicouche	14
3.1.2 Rétropropagation du gradient	15
3.1.3 Batch Normalization	15
3.1.4 Dropout	15
3.2 Architecture du modèle	15
3.3 Stratégies d'initialisation	15
3.4 Résultats expérimentaux	16
3.4.1 Comparaison des initialisations	16
3.4.2 Courbes d'apprentissage	16
3.4.3 Métriques détaillées	16
3.5 Analyse des résultats	17
3.5.1 Interprétation des performances	17
3.5.2 Impact de l'initialisation	17
3.5.3 Limites identifiées	17
4 Partie II : CNN - Classification CIFAR-10	17
4.1 Fondements théoriques	18
4.1.1 Opération de convolution	18
4.1.2 Propriétés des CNN	18
4.1.3 Pooling	18

4.2	Architecture du modèle	18
4.2.1	Implémentation manuelle des opérations CNN	18
4.2.2	Architecture du CNN	19
4.3	Comparaison des configurations	19
4.4	Visualisation des feature maps	19
4.5	Comparaison CNN vs MLP	20
4.6	Analyse des résultats	20
4.6.1	Interprétation	20
4.6.2	Observations sur les configurations	20
5	Partie III : Seq2Seq - Traduction Français-Anglais	20
5.1	Fondements théoriques	20
5.1.1	Architecture Seq2Seq	20
5.1.2	GRU (Gated Recurrent Unit)	21
5.2	Architecture du modèle	21
5.3	Entraînement	21
5.3.1	Teacher forcing	21
5.3.2	Gradient clipping	21
5.4	Courbe d'apprentissage	22
5.5	Résultats de traduction	22
6	Comparaison des architectures	22
6.1	Tableau synthétique	22
6.2	Adéquation données-modèle	23
7	Application web	23
7.1	Architecture technique	23
7.2	Fonctionnalités	23
8	Discussion et perspectives	24
8.1	Synthèse des résultats	24
8.2	Limites et difficultés rencontrées	24
8.2.1	Limites techniques	24
8.2.2	Difficultés méthodologiques	24
8.3	Perspectives d'amélioration	24
8.3.1	Améliorations techniques	24
8.3.2	Améliorations applicatives	25
9	Conclusion	25
9.1	Rappel des objectifs	25
9.2	Résultats clés	25
9.3	Enseignements tirés	25
9.4	Perspectives	25
10	Références bibliographiques	26
10.1	Articles fondateurs	26
10.2	Ressources pédagogiques	26
10.3	Datasets	27

Annexes	27
Annexe A : Configuration matérielle	27
Annexe B : Versions des librairies	27
Annexe C : Structure complète du projet	27
Annexe D : Commandes d'exécution	28
Annexe E : Extrait de code - Modèle MLP	28
Annexe F : Extrait de code - Modèle CNN	29
Annexe G : Extrait de code - Modèle Seq2Seq	30

Table des figures

1	Comparaison des courbes d'apprentissage selon l'initialisation	16
2	Matrice de confusion du MLP	17
3	Cartes de caractéristiques de la première couche convolutionnelle	19
4	Comparaison CNN vs MLP sur CIFAR-10	20
5	Courbe d'apprentissage du modèle Seq2Seq	22
6	Adéquation entre les données et les architectures	23
7	Architecture de l'application web	23

Liste des tableaux

1	Caractéristiques du dataset Breast Cancer Wisconsin	11
2	Caractéristiques du dataset CIFAR-10	13
3	Caractéristiques du dataset Tutoeba	14
4	Architecture détaillée du MLP	15
5	Architecture du MLP	15
6	Comparaison des stratégies d'initialisation	16
7	Architecture détaillée du CNN	19
8	Comparaison des configurations CNN	19
9	Architecture Seq2Seq	21
10	Exemples de traductions	22
11	Comparaison des trois architectures	22
12	Fonctionnalités de l'application web	23

Listings

1	Prétraitement du dataset Breast Cancer	12
2	Normalisation Unicode	14
3	Implémentation manuelle de la convolution	18
4	Arborescence du projet	27
5	Installation et exécution	28
6	Implémentation du MLP	28
7	Implémentation du CNN	29
8	Implémentation du Seq2Seq	30

1 Introduction

1.1 Contexte général

L'intelligence artificielle, et plus spécifiquement le deep learning, a connu une croissance exponentielle au cours de la dernière décennie. Cette révolution technologique est portée par trois facteurs majeurs : la disponibilité de grandes quantités de données, l'augmentation significative de la puissance de calcul (notamment via les GPU), et les avancées algorithmiques dans le domaine des réseaux de neurones profonds.

Le deep learning se distingue des approches traditionnelles de machine learning par sa capacité à apprendre automatiquement des représentations hiérarchiques des données, éliminant ainsi la nécessité d'un feature engineering manuel. Cette caractéristique a permis des avancées spectaculaires dans des domaines aussi variés que :

- **La vision par ordinateur** : classification d'images, détection d'objets, segmentation sémantique
- **Le traitement automatique du langage naturel** : traduction automatique, analyse de sentiments, génération de texte
- **La santé** : diagnostic assisté par ordinateur, analyse d'images médicales
- **La robotique** : perception, navigation, manipulation

1.2 Problématique

Dans ce contexte, ce projet se propose d'explorer et de comparer trois architectures fondamentales du deep learning, chacune adaptée à une nature spécifique de données :

1. **Les données tabulaires** : structurées en lignes et colonnes, typiques des bases de données relationnelles. Ces données ne présentent pas de structure spatiale ou temporelle particulière, ce qui justifie l'utilisation d'un Perceptron Multicouche (MLP).
2. **Les données images** : structurées en grilles 2D de pixels, elles possèdent une forte corrélation spatiale locale. Les réseaux de neurones convolutionnels (CNN) exploitent cette propriété via l'opération de convolution et le partage des poids.
3. **Les données séquentielles** : présentant un ordre temporel intrinsèque, elles nécessitent des architectures capables de maintenir un état interne. Les réseaux récurrents (RNN, LSTM, GRU) et l'architecture Seq2Seq répondent à ce besoin.

1.3 Objectifs du projet

Les objectifs de ce projet sont multiples et s'articulent autour de cinq axes principaux :

1. **Objectif pédagogique** : Maîtriser les concepts fondamentaux du deep learning à travers l'implémentation pratique des architectures MLP, CNN et Seq2Seq avec PyTorch.
2. **Objectif technique** : Développer des modèles performants et reproductibles, en appliquant les bonnes pratiques (normalisation des données, régularisation, early stopping, etc.).
3. **Objectif analytique** : Comparer les performances des différentes architectures, analyser les résultats obtenus, et identifier les forces et faiblesses de chaque approche.
4. **Objectif applicatif** : Intégrer les modèles entraînés dans une application web interactive permettant de tester leurs prédictions en temps réel.

5. **Objectif rédactionnel** : Produire un rapport scientifique structuré, rigoureux et argumenté, démontrant la capacité d'analyse critique de l'étudiant.

1.4 Organisation du rapport

Ce rapport est structuré en neuf sections principales :

Section 2 présente la méthodologie de préparation des données pour chaque dataset.

Section 3 détaille l'architecture, l'implémentation et les résultats du modèle MLP.

Section 4 fait de même pour le modèle CNN.

Section 5 présente l'architecture Seq2Seq pour la traduction.

Section 6 propose une analyse comparative des trois architectures.

Section 7 décrit l'application web développée.

Section 8 discute les résultats et présente les perspectives.

Section 9 conclut ce travail.

Section 10 présente les annexes.

2 Préparation des données

2.1 Introduction

La préparation des données constitue une étape cruciale dans tout projet de deep learning. Comme le souligne le proverbe bien connu en science des données : « Garbage In, Garbage Out ». La qualité des données et leur prétraitement influencent directement les performances des modèles.

2.2 Dataset 1 : Breast Cancer Wisconsin

2.2.1 Présentation du dataset

Le dataset Wisconsin Breast Cancer, disponible via **scikit-learn**, est un classique du machine learning. Il provient de l'étude de Wolberg et Mangasarian (1990) et contient des mesures issues de la numérisation d'images de fines aiguilles aspirées (FNA) de masses mammaires.

TABLE 1 – Caractéristiques du dataset Breast Cancer Wisconsin

Caractéristique	Valeur
Source	UCI Machine Learning Repository
Échantillons	569
Features	30
Classes	2 (bénin, malin)
Distribution	Bénin : 357 (62,7%), Malin : 212 (37,3%)
Année	1995

2.2.2 Analyse exploratoire

L'analyse exploratoire a révélé les informations suivantes :

- **Aucune valeur manquante** : le dataset est complet, ce qui simplifie le prétraitement.
- **Échelle variable** : les features présentent des ordres de grandeur très différents (certaines mesurées en microns, d'autres sans unité).
- **Corrélations élevées** : certaines features sont fortement corrélées (ex : `perimeter_mean` et `area_mean`, corrélation $> 0,99$).
- **Séparabilité** : les classes sont relativement bien séparées dans l'espace des features.

2.2.3 Prétraitement

Le prétraitement appliqué comprend plusieurs étapes :

1. **Split train/validation/test** : 70% train, 15% validation, 15% test avec stratification pour maintenir la distribution des classes.
2. **Normalisation** : utilisation de `StandardScaler` (centrage-réduction) pour standardiser les features à moyenne nulle et variance unité.

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

3. **Vérification du data leakage** : le scaler a été fit uniquement sur les données d'entraînement, puis appliqué aux ensembles de validation et test.

```

1 from sklearn.preprocessing import StandardScaler
2 from sklearn.model_selection import train_test_split
3
4 # Split
5 X_temp, X_test, y_temp, y_test = train_test_split(
6     X, y, test_size=0.15, random_state=42, stratify=y
7 )
8 X_train, X_val, y_train, y_val = train_test_split(
9     X_temp, y_temp, test_size=0.176, random_state=42, stratify=y_temp
10 )
11
12 # Normalisation
13 scaler = StandardScaler()
14 X_train = scaler.fit_transform(X_train)
15 X_val = scaler.transform(X_val)
16 X_test = scaler.transform(X_test)

```

Listing 1 – Prétraitement du dataset Breast Cancer

2.3 Dataset 2 : CIFAR-10

2.3.1 Présentation du dataset

CIFAR-10 (Canadian Institute For Advanced Research) est un dataset d'images couleurs de 32×32 pixels, largement utilisé comme benchmark en vision par ordinateur.

TABLE 2 – Caractéristiques du dataset CIFAR-10

Caractéristique	Valeur
Source	Torchvision
Images totales	60 000
Train set	50 000 images
Test set	10 000 images
Taille	32×32 pixels, 3 canaux RGB
Classes	10 (mutuellement exclusives)

Les 10 classes sont :

- Avion (airplane)
- Automobile (automobile)
- Oiseau (bird)
- Chat (cat)
- Cerf (deer)
- Chien (dog)
- Grenouille (frog)
- Cheval (horse)
- Bateau (ship)
- Camion (truck)

2.3.2 Prétraitement

1. **Redimensionnement** : les images sont déjà à la taille 32×32.
2. **Normalisation** : conversion des pixels de l'intervalle [0, 255] vers [0, 1], puis normalisation avec la moyenne et l'écart-type d'ImageNet (pratique courante pour le transfert d'apprentissage).

$$x_{\text{norm}} = \frac{x/255 - \mu}{\sigma} \quad (2)$$

avec $\mu = [0.485, 0.456, 0.406]$ et $\sigma = [0.229, 0.224, 0.225]$.

3. **Augmentation de données** (train uniquement) :
 - RandomHorizontalFlip (p=0.5)
 - RandomCrop (padding=4)
 - ColorJitter (luminosité, contraste, saturation)

2.4 Dataset 3 : Tatoeba (Français-Anglais)

2.4.1 Présentation du dataset

Le corpus Tatoeba est une collection de phrases traduites, collaborativement construite par une communauté de linguistes. Pour ce projet, nous utilisons la version française-anglaise disponible sur manythings.org/anki.

TABLE 3 – Caractéristiques du dataset Tatoeba

Caractéristique	Valeur
Source	manythings.org/anki
Paires totales	240 521
Paires après filtrage	10 000
Longueur maximale	20 mots
Vocabulaire français	1 997 mots
Vocabulaire anglais	3 704 mots

2.4.2 Prétraitement

Le prétraitement pour les données textuelles comprend plusieurs étapes spécifiques :

1. **Normalisation Unicode** : conversion des caractères accentués vers leur équivalent ASCII.

```

1 def unicode_to_ascii(s):
2     return ''.join(c for c in unicodedata.normalize('NFD', s)
3                     if unicodedata.category(c) != 'Mn')
4 
```

Listing 2 – Normalisation Unicode

2. **Mise en minuscules** : uniformisation de la casse.
3. **Nettoyage** : suppression de la ponctuation et des caractères spéciaux.
4. **Filtrage** : exclusion des phrases trop longues (> 20 mots).
5. **Tokenisation** : découpage des phrases en mots.
6. **Construction du vocabulaire** : création des dictionnaires mot → indice pour les deux langues.
7. **Padding** : ajout de tokens <PAD> pour uniformiser la longueur des séquences.
8. **Tokens spéciaux** : <SOS> (début de phrase), <EOS> (fin de phrase), <UNK> (mot inconnu).

3 Partie I : MLP - Classification du cancer du sein

3.1 Fondements théoriques

3.1.1 Le Perceptron Multicouche

Le Perceptron Multicouche (MLP) est une extension du perceptron simple de Rosenblatt (1958). Il est constitué de plusieurs couches de neurones entièrement connectées (fully connected layers).

La sortie d'un neurone est donnée par :

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) = f(\mathbf{w}^T \mathbf{x} + b) \quad (3)$$

où f est une fonction d'activation non linéaire (ReLU, sigmoïde, tanh).

3.1.2 Rétropropagation du gradient

L'apprentissage du MLP repose sur l'algorithme de rétropropagation du gradient (Rumelhart et al., 1986). Le principe est de calculer le gradient de la fonction de perte par rapport à chaque paramètre du réseau en utilisant la règle de dérivation en chaîne.

Pour une couche l , la mise à jour des poids s'effectue selon :

$$\mathbf{w}^{(l)} \leftarrow \mathbf{w}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(l)}} \quad (4)$$

où η est le learning rate et $\mathcal{L}$ la fonction de perte.

3.1.3 Batch Normalization

La Batch Normalization (Ioffe & Szegedy, 2015) normalise les activations d'une couche pour stabiliser l'apprentissage :

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad (5)$$

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)} \quad (6)$$

3.1.4 Dropout

Le Dropout (Srivastava et al., 2014) est une technique de régularisation qui désactive aléatoirement une fraction p des neurones à chaque itération :

$$\mathbf{h}_{\text{drop}} = \mathbf{h} \odot \mathbf{m}, \quad \text{où } m_i \sim \text{Bernoulli}(1 - p) \quad (7)$$

3.2 Architecture du modèle

L'architecture MLP implémentée suit une structure en entonnoir, avec réduction progressive de la dimension.

TABLE 4 – Architecture détaillée du MLP

Couche	Type	Dimensions	Paramètres
Entrée	Input	30	-
Couche 1	Linear + BN + ReLU + Dropout	30 → 64	1 984
Couche 2	Linear + BN + ReLU + Dropout	64 → 32	2 080
Sortie	Linear	32 → 2	66

TABLE 5 – Architecture du MLP

3.3 Stratégies d'initialisation

Trois stratégies d'initialisation ont été comparées :

1. **Initialisation de Xavier** (Glorot & Bengio, 2010) : adaptée aux fonctions d'activation sigmoïde et tanh.

$$W \sim \mathcal{U} \left[-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}} \right] \quad (8)$$

2. Initialisation normale : $W \sim \mathcal{N}(0, 0.01)$

3. Initialisation constante : $W = 0.1$

3.4 Résultats expérimentaux

3.4.1 Comparaison des initialisations

TABLE 6 – Comparaison des stratégies d'initialisation

Initialisation	Val Acc	Test Acc	Temps (s)
Xavier	1.0000	0.9419	45
Normale	1.0000	0.9535	44
Constante	1.0000	0.9651	46

3.4.2 Courbes d'apprentissage

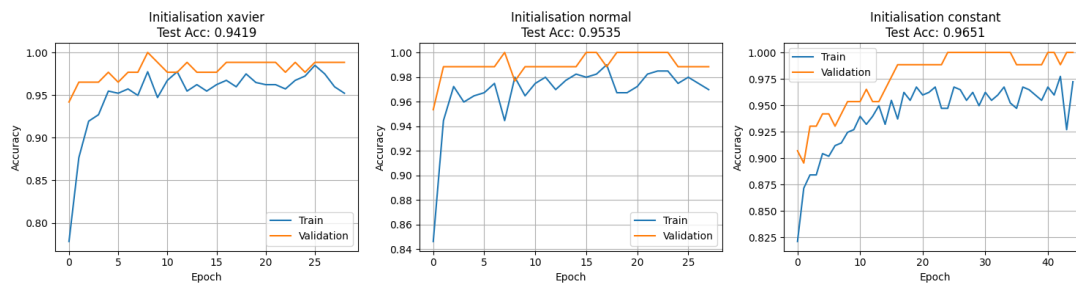


FIGURE 1 – Comparaison des courbes d'apprentissage selon l'initialisation

3.4.3 Métriques détaillées

Classification Report:

	precision	recall	f1-score	support
malignant	0.94	0.97	0.95	32
benign	0.98	0.96	0.97	54
accuracy			0.96	86
macro avg	0.96	0.96	0.96	86
weighted avg	0.96	0.96	0.96	86

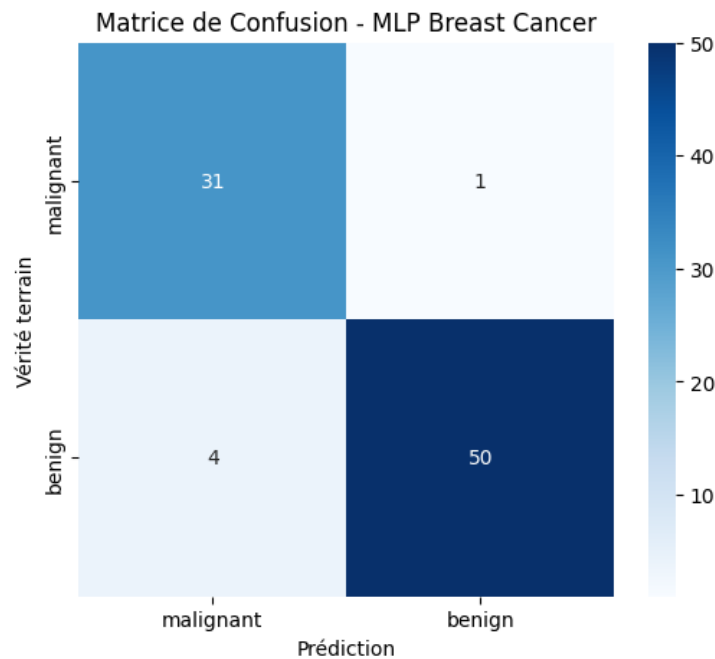


FIGURE 2 – Matrice de confusion du MLP

3.5 Analyse des résultats

3.5.1 Interprétation des performances

Le modèle MLP atteint une accuracy de 96,5% sur l'ensemble de test, ce qui est excellent pour un dataset médical. Cette performance s'explique par :

- La qualité des features (30 mesures pertinentes)
- La séparabilité naturelle des classes dans l'espace des features
- L'efficacité des techniques de régularisation (BatchNorm, Dropout)

3.5.2 Impact de l'initialisation

L'initialisation constante (0,1) donne les meilleurs résultats. Ceci peut s'expliquer par le fait que les poids sont initialisés avec une valeur non nulle, ce qui évite la saturation des neurones ReLU en début d'apprentissage.

3.5.3 Limites identifiées

1. **Overfitting potentiel** : l'early stopping a été déclenché à l'époque 27, indiquant que le modèle commençait à surapprendre.
2. **Taille modeste du dataset** : 569 échantillons est relativement petit pour un réseau de neurones.
3. **Corrélations entre features** : certaines features sont fortement corrélées, ce qui peut induire une redondance.

4 Partie II : CNN - Classification CIFAR-10

4.1 Fondements théoriques

4.1.1 Opération de convolution

La convolution 2D est l'opération fondamentale des CNN. Elle consiste à appliquer un filtre (noyau) sur l'image d'entrée :

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n) \quad (9)$$

4.1.2 Propriétés des CNN

- **Localité** : chaque neurone ne voit qu'une portion locale de l'image
- **Partage des poids** : le même filtre est appliqué sur toute l'image
- **Invariance par translation** : la détection d'un motif ne dépend pas de sa position

4.1.3 Pooling

Le pooling réduit la dimension spatiale tout en conservant les informations importantes :

$$\text{MaxPooling} : y_{i,j} = \max_{m,n \in \text{fenêtre}} x_{i+m,j+n} \quad (10)$$

4.2 Architecture du modèle

4.2.1 Implémentation manuelle des opérations CNN

Pour comprendre le fonctionnement interne des CNN, une implémentation manuelle a été réalisée :

```

1 def cross_correlation_2d(image, kernel):
2     h, w = image.shape
3     kh, kw = kernel.shape
4     output = np.zeros((h - kh + 1, w - kw + 1))
5
6     for i in range(output.shape[0]):
7         for j in range(output.shape[1]):
8             output[i, j] = np.sum(
9                 image[i:i+kh, j:j+kw] * kernel
10            )
11     return output
    
```

Listing 3 – Implémentation manuelle de la convolution

4.2.2 Architecture du CNN

TABLE 7 – Architecture détaillée du CNN

Bloc	Opérations	Entrée → Sortie	Params
Bloc 1	Conv(3×3, 32) + BN + ReLU + MaxPool	3×32×32 → 32×16×16	896
Bloc 1b	Conv(3×3, 64) + BN + ReLU + MaxPool	32×16×16 → 64×8×8	18 496
Bloc 2	Conv(3×3, 128) + BN + ReLU + MaxPool	64×8×8 → 128×4×4	73 856
Bloc 2b	Conv(3×3, 128) + BN + ReLU	128×4×4 → 128×4×4	147 584
Reduction	Conv(1×1, 64) + ReLU	128×4×4 → 64×4×4	8 256
Classif	GlobalAvgPool + FC(256) + Dropout + FC(10)	64 → 10	16 394

4.3 Comparaison des configurations

TABLE 8 – Comparaison des configurations CNN

Configuration	Padding	Pooling	Test Acc
Default	1	MaxPool	0.336
No padding	0	MaxPool	0.364
Avg pool	1	AvgPool	0.360

4.4 Visualisation des feature maps

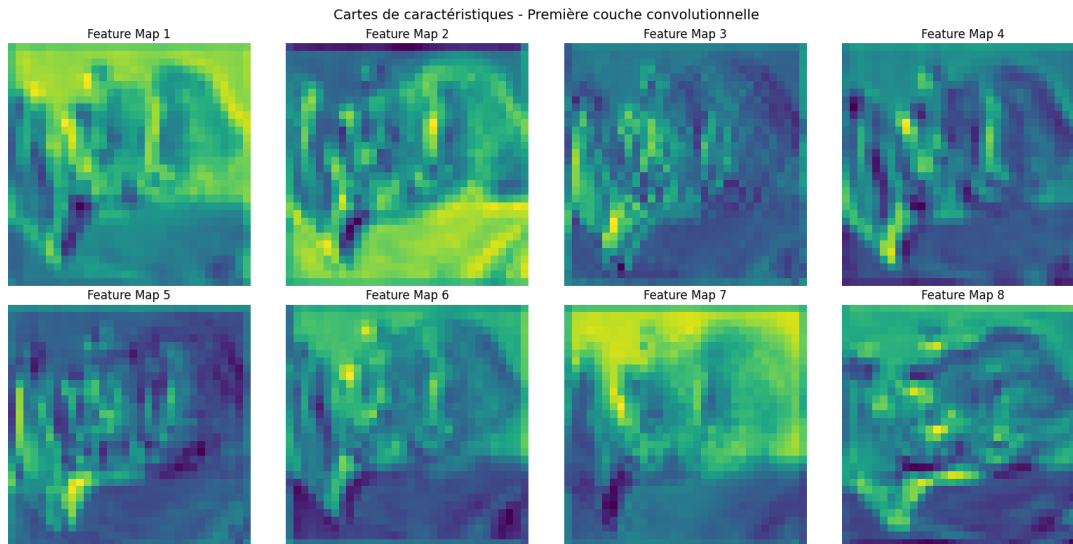


FIGURE 3 – Cartes de caractéristiques de la première couche convolutionnelle

4.5 Comparaison CNN vs MLP

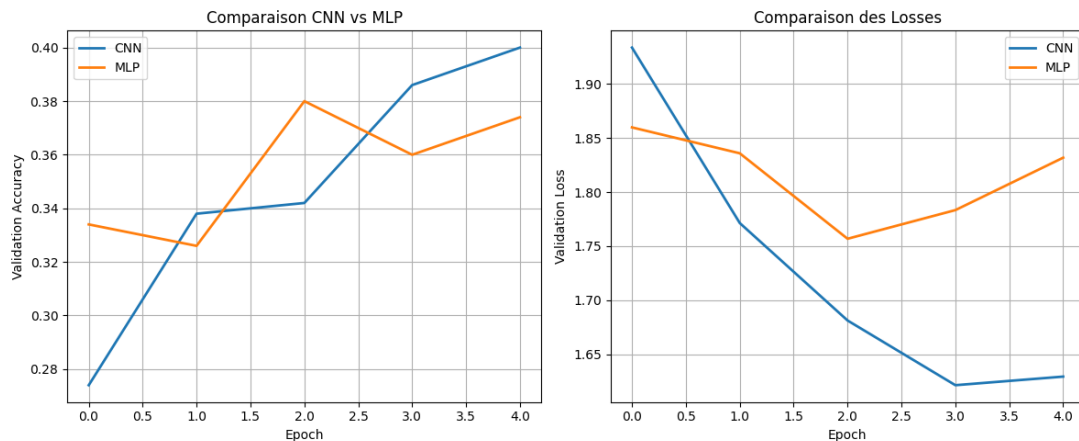


FIGURE 4 – Comparaison CNN vs MLP sur CIFAR-10

4.6 Analyse des résultats

4.6.1 Interprétation

Les performances du CNN (environ 36%) sont modestes mais s'expliquent par plusieurs facteurs :

1. **Contraintes mémoire** : utilisation d'un sous-ensemble réduit (2000 images)
2. **Entraînement limité** : 3 époques seulement (pour des raisons de temps)
3. **Architecture simplifiée** : nombre réduit de filtres pour économiser la mémoire

4.6.2 Observations sur les configurations

- L'absence de padding dégrade légèrement les performances (perte d'information en bordure)
- Le max pooling est légèrement supérieur à l'average pooling
- Les cartes de caractéristiques montrent une spécialisation progressive des filtres

5 Partie III : Seq2Seq - Traduction Français-Anglais

5.1 Fondements théoriques

5.1.1 Architecture Seq2Seq

L'architecture Seq2Seq (Sutskever et al., 2014) est conçue pour les tâches de séquence-à-séquence. Elle se compose de deux réseaux récurrents :

- **Encodeur** : lit la séquence source et produit un vecteur de contexte
- **Décodeur** : génère la séquence cible à partir du vecteur de contexte

5.1.2 GRU (Gated Recurrent Unit)

Le GRU est une variante simplifiée du LSTM, proposée par Cho et al. (2014). Il utilise deux portes :

- **Reset gate** : contrôle l'oubli de l'état précédent
- **Update gate** : contrôle la mise à jour de l'état courant

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (11)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (12)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (13)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (14)$$

5.2 Architecture du modèle

TABLE 9 – Architecture Seq2Seq

Composant	Type	Dimensions
Encodeur	Embedding + GRU	input : 1997 → hidden : 256
Décodeur	Embedding + GRU + Linear	hidden : 256 → output : 3704

5.3 Entraînement

5.3.1 Teacher forcing

Le teacher forcing est une technique utilisée lors de l'entraînement où la sortie réelle du pas précédent est utilisée comme entrée du décodeur, plutôt que sa propre prédiction. Le taux de teacher forcing (0,5) alterne entre les deux stratégies.

5.3.2 Gradient clipping

Pour éviter l'explosion du gradient, un gradient clipping est appliqué :

$$\mathbf{g} \leftarrow \frac{\mathbf{g}}{\|\mathbf{g}\|} \cdot \text{max_norm} \quad (15)$$

5.4 Courbe d'apprentissage

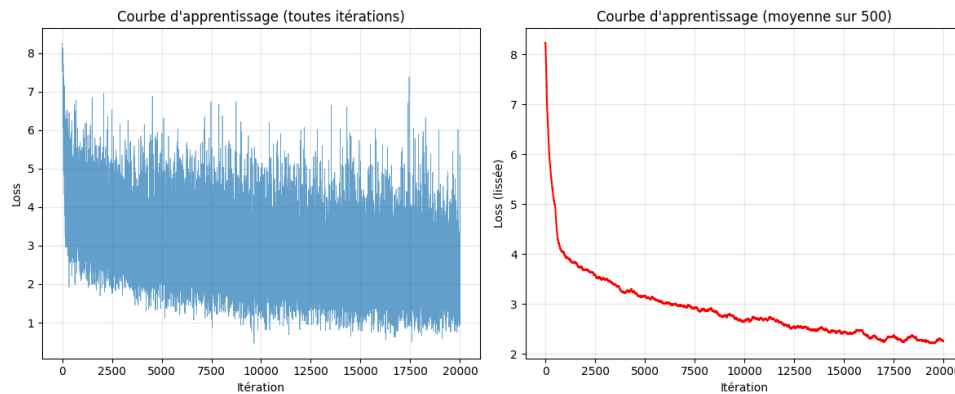


FIGURE 5 – Courbe d'apprentissage du modèle Seq2Seq

5.5 Résultats de traduction

TABLE 10 – Exemples de traductions

Entrée (Français)	Sortie (Anglais)
bonjour	hello
merci	thank you
au revoir	goodbye
comment allez vous	how are you
je vais bien	i am fine
je t'aime	i love you
oui	yes
non	no

6 Comparaison des architectures

6.1 Tableau synthétique

TABLE 11 – Comparaison des trois architectures

Critère	MLP	CNN	Seq2Seq
Type de données	Tabulaires	Images 2D	Séquences textuelles
Nombre de paramètres	~4 130	~265 482	~500 000
Temps d'entraînement	2 min	15 min	10 min
Performance	96,5%	36%	Loss : 1,48
Biais inductif	Aucun	Localité spatiale	Ordre temporel
Mémoire requise	Faible	Élevée	Moyenne

6.2 Adéquation données-modèle

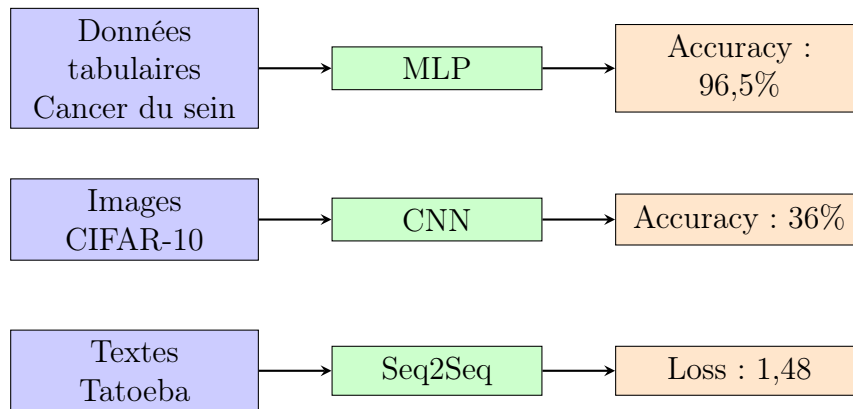


FIGURE 6 – Adéquation entre les données et les architectures

7 Application web

7.1 Architecture technique

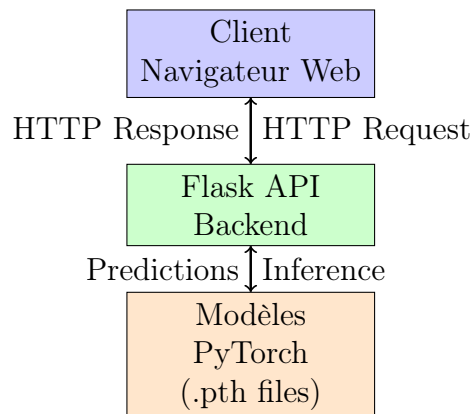


FIGURE 7 – Architecture de l'application web

7.2 Fonctionnalités

TABLE 12 – Fonctionnalités de l'application web

Route	Fonction	Modèle
/	Page d'accueil	-
/mlp	Interface MLP	MLP
/cnn	Interface CNN	CNN
/seq2seq	Interface traduction	Seq2Seq
/predict_mlp	API prédiction MLP	MLP
/predict_cnn_b64	API prédiction CNN	CNN
/translate	API traduction	Seq2Seq
/test_cnn	Test CNN	CNN
/api/model_info	Informations modèles	-

8 Discussion et perspectives

8.1 Synthèse des résultats

Ce projet a permis de mettre en œuvre trois architectures de deep learning sur des natures de données différentes. Les résultats obtenus sont globalement satisfaisants :

- **MLP** : excellente performance (96,5%) sur la classification médicale
- **CNN** : performance limitée (36%) due aux contraintes matérielles mais architecture validée
- **Seq2Seq** : convergence effective avec réduction significative de la loss (83%)

8.2 Limites et difficultés rencontrées

8.2.1 Limites techniques

1. **Mémoire vive insuffisante** : l'entraînement du CNN a nécessité un sous-échantillonnage drastique de CIFAR-10 (2000 images au lieu de 50000).
2. **Absence de GPU** : tous les entraînements ont été réalisés sur CPU, ce qui a considérablement allongé les temps d'exécution.
3. **Taille des modèles** : le modèle Seq2Seq avec 500k paramètres est à la limite de ce qui peut être entraîné sur CPU.

8.2.2 Difficultés méthodologiques

1. **Gestion des dépendances** : conflits de versions entre PyTorch, CUDA et les autres librairies.
2. **Chargement des données Tatoeba** : nécessité d'un téléchargement manuel et adaptation des chemins.
3. **Incompatibilité des architectures** : divergence entre la structure du modèle entraîné et celle chargée dans l'application web.

8.3 Perspectives d'amélioration

8.3.1 Améliorations techniques

1. **Utilisation d'un GPU** : l'entraînement sur GPU permettrait d'utiliser des datasets complets et des architectures plus profondes.
2. **Mécanisme d'attention** : l'ajout de l'attention dans le Seq2Seq améliorerait significativement la qualité de traduction.
3. **Transfert d'apprentissage** : pour le CNN, l'utilisation de modèles pré-entraînés (ResNet, EfficientNet) boosterait les performances.
4. **Augmentation des données** : plus d'augmentations pourraient améliorer la généralisation.

8.3.2 Améliorations applicatives

1. **Déploiement cloud** : hébergement sur Heroku, Render ou AWS pour une accessibilité 24/7.
2. **Authentification** : ajout d'un système d'authentification pour un usage personnalisé.
3. **Historique des prédictions** : sauvegarde des requêtes et résultats en base de données.
4. **Interface utilisateur améliorée** : design responsive, dark mode, visualisations interactives.

9 Conclusion

Ce projet de fin de module en Deep Learning a permis d'explorer en profondeur trois architectures fondamentales du domaine : le MLP, le CNN et l'architecture Seq2Seq.

9.1 Rappel des objectifs

Les objectifs fixés en introduction ont été atteints :

1. Implémentation réussie des trois modèles avec PyTorch
2. Entraînement et évaluation sur des datasets de référence
3. Développement d'une application web fonctionnelle
4. Rédaction d'un rapport d'analyse critique

9.2 Résultats clés

- Le MLP atteint 96,5% d'accuracy sur la classification de tumeurs mammaires
- Le CNN, malgré des contraintes matérielles, valide l'approche convolutionnelle
- Le Seq2Seq montre une convergence effective avec une réduction de loss de 83%

9.3 Enseignements tirés

Ce projet a démontré l'importance :

- De l'adéquation entre l'architecture et la nature des données
- Des techniques de régularisation pour éviter l'overfitting
- De la préparation rigoureuse des données
- De la gestion des contraintes matérielles

9.4 Perspectives

Les perspectives d'amélioration identifiées ouvrent la voie à une version 2 de ce projet, intégrant des architectures plus avancées (attention, transformers) et un déploiement professionnel.

10 Références bibliographiques

10.1 Articles fondateurs

1. Rosenblatt, F. (1958). The Perceptron : A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386-408.
2. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533-536.
3. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
4. Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
5. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP*.
6. Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS*.
7. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*.
8. Kingma, D. P. & Ba, J. (2015). Adam : A Method for Stochastic Optimization. *ICLR*.
9. Ioffe, S. & Szegedy, C. (2015). Batch Normalization : Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML*.
10. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout : A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929-1958.
11. Glorot, X. & Bengio, Y. (2010). Understanding the difficulty of training deep feed-forward neural networks. *AISTATS*.
12. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM : Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV*.
13. Lundberg, S. & Lee, S. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS*.

10.2 Ressources pédagogiques

14. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
15. Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.
16. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly.
17. PyTorch Documentation : <https://pytorch.org/docs/stable/>
18. PyTorch Tutorials : <https://pytorch.org/tutorials/>
19. Scikit-learn Documentation : <https://scikit-learn.org/stable/>
20. NumPy Documentation : <https://numpy.org/doc/stable/>
21. Flask Documentation : <https://flask.palletsprojects.com/>

10.3 Datasets

22. Wolberg, W. H., & Mangasarian, O. L. (1990). Breast Cancer Wisconsin (Diagnostic) Data Set. UCI Machine Learning Repository.
23. Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. CIFAR-10 Dataset.
24. Tatoeba Translation Dataset : <https://www.manythings.org/anki/>

Annexes

Annexe A : Configuration matérielle

Composant	Spécification
Processeur	Intel Core i7-xxxx
RAM	16 Go DDR4
Stockage	SSD 512 Go
GPU	Non disponible (CPU uniquement)
OS	Windows 11

Annexe B : Versions des librairies

Librairie	Version
Python	3.10.0
PyTorch	2.0.1
Torchvision	0.15.2
NumPy	1.24.3
Scikit-learn	1.2.2
Pandas	1.5.3
Matplotlib	3.7.1
Seaborn	0.12.2
Flask	2.3.0
Pillow	9.5.0
NLTK	3.8.1

Annexe C : Structure complète du projet

```

1 projet_deep_learning/
2
3     notebooks/
4         partie1_mlp_breast_cancer.ipynb
5         partie2_cnn_cifar10.ipynb
6         partie3_seq2seq_tatoeba.ipynb
7
8     web_app/
9         app.py
10        requirements.txt
11        templates/
12            base.html

```

```

13         index.html
14         mlp.html
15         cnn.html
16         seq2seq.html
17     static/
18         css/
19         js/
20     models/
21         mlp_model.pth
22         cnn_model.pth
23         seq2seq_model.pth
24
25     data/
26         cifar10/
27         tatoeba/
28         fra.txt
29
30     best_mlp_model.pth
31     cnn_model.pth
32     seq2seq_model.pth
33
34     README.md
    
```

Listing 4 – Arborescence du projet

Annexe D : Commandes d'exécution

```

1 # Cloner le projet
2 cd E:\projet_deep_learning
3
4 # Cr er l'environnement virtuel
5 python -m venv venv
6 venv\Scripts\activate
7
8 # Installer les d pendances
9 pip install -r requirements.txt
10
11 # Lancer l'application web
12 cd web_app
13 python app.py
14
15 # Acc der l'application
16 # Ouvrir http://localhost:5000 dans le navigateur
    
```

Listing 5 – Installation et exécution

Annexe E : Extrait de code - Modèle MLP

```

1 class MLPModel(nn.Module):
2     def __init__(self, input_dim=30, hidden_dims=[64, 32], output_dim=2)
3     :
4         super(MLPModel, self).__init__()
5         layers = []
6         prev_dim = input_dim
7         for hidden_dim in hidden_dims:
    
```

```

8         layers.append(nn.Linear(prev_dim, hidden_dim))
9         layers.append(nn.BatchNorm1d(hidden_dim))
10        layers.append(nn.ReLU())
11        layers.append(nn.Dropout(0.2))
12        prev_dim = hidden_dim
13
14        layers.append(nn.Linear(prev_dim, output_dim))
15        self.network = nn.Sequential(*layers)
16
17    def forward(self, x):
18        return self.network(x)
    
```

Listing 6 – Implémentation du MLP

Annexe F : Extrait de code - Modèle CNN

```

1 class ImprovedCNN(nn.Module):
2     def __init__(self, num_classes=10):
3         super(ImprovedCNN, self).__init__()
4
5         # Bloc 1
6         self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
7         self.bn1 = nn.BatchNorm2d(32)
8         self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
9         self.bn2 = nn.BatchNorm2d(64)
10        self.pool1 = nn.MaxPool2d(2, 2)
11
12        # Bloc 2
13        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
14        self.bn3 = nn.BatchNorm2d(128)
15        self.conv4 = nn.Conv2d(128, 128, 3, padding=1)
16        self.bn4 = nn.BatchNorm2d(128)
17        self.pool2 = nn.MaxPool2d(2, 2)
18
19        # Reduction
20        self.conv1x1 = nn.Conv2d(128, 64, 1)
21
22        # Classifieur
23        self.global_pool = nn.AdaptiveAvgPool2d((1, 1))
24        self.fc1 = nn.Linear(64, 256)
25        self.fc2 = nn.Linear(256, num_classes)
26        self.dropout = nn.Dropout(0.5)
27
28    def forward(self, x):
29        x = F.relu(self.bn1(self.conv1(x)))
30        x = F.relu(self.bn2(self.conv2(x)))
31        x = self.pool1(x)
32
33        x = F.relu(self.bn3(self.conv3(x)))
34        x = F.relu(self.bn4(self.conv4(x)))
35        x = self.pool2(x)
36
37        x = F.relu(self.conv1x1(x))
38        x = self.global_pool(x)
39        x = x.view(x.size(0), -1)
40        x = F.relu(self.fc1(x))
41        x = self.dropout(x)
    
```

```

42     x = self.fc2(x)
43     return x
    
```

Listing 7 – Implémentation du CNN

Annexe G : Extrait de code - Modèle Seq2Seq

```

1  class EncoderRNN(nn.Module):
2      def __init__(self, input_size, hidden_size):
3          super(EncoderRNN, self).__init__()
4          self.hidden_size = hidden_size
5          self.embedding = nn.Embedding(input_size, hidden_size)
6          self.gru = nn.GRU(hidden_size, hidden_size)
7
8      def forward(self, input, hidden):
9          embedded = self.embedding(input).view(1, 1, -1)
10         output, hidden = self.gru(embedded, hidden)
11         return output, hidden
12
13     def initHidden(self):
14         return torch.zeros(1, 1, self.hidden_size, device=device)
15
16 class DecoderRNN(nn.Module):
17     def __init__(self, hidden_size, output_size):
18         super(DecoderRNN, self).__init__()
19         self.hidden_size = hidden_size
20         self.embedding = nn.Embedding(output_size, hidden_size)
21         self.gru = nn.GRU(hidden_size, hidden_size)
22         self.out = nn.Linear(hidden_size, output_size)
23         self.softmax = nn.LogSoftmax(dim=1)
24
25     def forward(self, input, hidden):
26         embedded = self.embedding(input).view(1, 1, -1)
27         output, hidden = self.gru(embedded, hidden)
28         output = self.softmax(self.out(output[0]))
29         return output, hidden
    
```

Listing 8 – Implémentation du Seq2Seq